

AI ASSISTED CODING

FINAL EXAM

NAME: G.Karthik Reddy

ROLL

NO: 2403A510f8

BATCH: 06

DATE:24-11-2025

SUBSET:10 - Refactoring Legacy Code

Q1: Clean up old attendance module

- Task 1: Use AI to identify redundant conditions.
- Task 2: Refactor into modular functions.

PROMPT:

Analyse this attendance module code, identify redundant conditions, and refactor it into clean, modular functions while preserving functionality. Provide optimized and well-commented code.

CODE:

```
attendance.py •
attendance.py > normalize_status
1  students = [
2      {"id": 1, "name": "Alice", "active": True},
3      {"id": 2, "name": "Bob", "active": True},
4      {"id": 3, "name": "Charlie", "active": False},
5      {"id": 4, "name": "Diana", "active": True},
6  ]
7
8  attendance_records = {}
9  def find_student_by_id(student_id):
10     for s in students:
11         if s["id"] == student_id:
12             return s
13     return None
14  def is_valid_status(status):
15     # Intentionally redundant and messy condition block
16     if status == "p" or status == "A" or status == "L":
17         return True
18     elif status == "p" or status == "a" or status == "l":
19         return True
20     elif status.lower() == "present" or status.lower() == "absent" or status.lower() == "late"
21         return True
22     elif status.upper() == "p" or status.upper() == "A" or status.upper() == "L":
23         return True
24     else:
25         return False
26  def normalize_status(status):
27     # More redundancy
28     if status == "p" or status == "p" or status.lower() == "present":
29         return "p"
30     elif status == "A" or status == "a" or status.lower() == "absent":
31         return "A"
32     elif status == "L" or status == "l" or status.lower() == "late":
33         return "L"
34     else:
35         return status
36  def mark_attendance(date, stu, student_id, status):
```

```
attendance.py •
attendance.py > normalize_status
36 def mark_attendance(date_str, student_id, status):
37     # Redundant existence checks
38     student = find_student_by_id(student_id)
39     if student is None or student == None:
40         print("Student not found!")
41         return
42
43     if student["active"] == False or student["active"] is False:
44         print("Student is not active. Cannot mark attendance.")
45         return
46
47     if not is_valid_status(status):
48         print("Invalid status entered. Use P/A/L or Present/Absent/Late.")
49         return
50
51     status_norm = normalize_status(status)
52
53     if date_str not in attendance_records.keys():
54         attendance_records[date_str] = {}
55
56     # Redundant dict checks
57     if student_id in attendance_records[date_str]:
58         if attendance_records[date_str][student_id] == status_norm:
59             print("Same status already marked. Nothing to change.")
60         else:
61             attendance_records[date_str][student_id] = status_norm
62             print("Attendance updated for student:", student["name"])
63     else:
64         attendance_records[date_str][student_id] = status_norm
65         print("Attendance marked for student:", student["name"])
66
67
68 def get_attendance_for_date(date_str):
69     if date_str in attendance_records:
70         return attendance_records[date_str]
```

```
71     else:
72         return {}
73
74
75 def print_attendance_for_date(date_str):
76     records = get_attendance_for_date(date_str)
77     if records == {} or len(records.keys()) == 0:
78         print("No attendance found for date:", date_str)
79         return
80
81     print("Attendance for", date_str)
82     print("-----")
83     for s in students:
84         sid = s["id"]
85         if sid in records:
86             status = records[sid]
87         else:
88             status = "N/A" # Not marked
89
90         # Redundant active checks
91         if s["active"] == True or s["active"] is True:
92             active_label = "Active"
93         else:
94             if s["active"] == False:
95                 active_label = "Inactive"
96             else:
97                 active_label = "Inactive"
98
99         print(f"ID: {sid} | Name: {s['name']} | Status: {status} | {active_label}")
100
101
102 def print_summary_for_student(student_id):
103     student = find_student_by_id(student_id)
104     if student is None:
```

```

102 def print_summary_for_student(student_id):
103     student = find_student_by_id(student_id)
104     if student is None:
105         print("Student not found.")
106         return
107
108     total_p = 0
109     total_a = 0
110     total_l = 0
111
112     for date_str in attendance_records:
113         if student_id in attendance_records[date_str]:
114             st = attendance_records[date_str][student_id]
115             if st == "P":
116                 total_p = total_p + 1
117             elif st == "A":
118                 total_a = total_a + 1
119             elif st == "L":
120                 total_l = total_l + 1
121             else:
122                 # unnecessary else
123                 total_p = total_p + 0
124
125     print("Summary for:", student["name"])
126     print("Present:", total_p)
127     print("Absent:", total_a)
128     print("Late:", total_l)
129     # Redundant total
130     total_classes = total_p + total_a + total_l
131     if total_classes > 0:
132         percentage = (total_p / total_classes) * 100
133         if percentage >= 75:
134             remark = "GOOD"
135         elif percentage < 75 and percentage >= 50:
136             remark = "AVERAGE"

```

```

attendance.py •
attendance.py > normalize_status
102 def print_summary_for_student(student_id):
136         remark = "AVERAGE"
137     elif percentage < 50:
138         remark = "POOR"
139     else:
140         remark = "N/A"
141     print("Total Classes:", total_classes)
142     print("Attendance %:", round(percentage, 2))
143     print("Remark:", remark)
144 else:
145     print("No classes attended yet.")
146
147
148 def print_all_students():
149     print("Student List")
150     print("-----")
151     for s in students:
152         # Again redundant active checks
153         if s["active"] == True:
154             status = "Active"
155         elif s["active"] == False:
156             status = "Inactive"
157         else:
158             status = "Inactive"
159         print(f"ID: {s['id']} | Name: {s['name']} | {status}")
160
161
162 def main_menu():
163     # Big, non-modular, messy menu function
164     while True:
165         print("\n=== ATTENDANCE MODULE (OLD) ===")
166         print("1. Show all students")
167         print("2. Mark attendance")
168         print("3. View attendance for a date")
169         print("4. View summary for a student")
170         print("\n> ")

```

```
attendance.py •
attendance.py > normalize_status
162 def main_menu():
170     print("5. Exit")
171
172     choice = input("Enter your choice: ")
173
174     if choice == "1":
175         print_all_students()
176     elif choice == "2":
177         date_str = input("Enter date (YYYY-MM-DD): ")
178         try:
179             sid_str = input("Enter student ID: ")
180             sid = int(sid_str)
181         except:
182             print("Invalid student ID!")
183             continue
184         status = input("Enter status (P/A/L or Present/Absent/Late): ")
185         mark_attendance(date_str, sid, status)
186     elif choice == "3":
187         date_str = input("Enter date (YYYY-MM-DD): ")
188         print_attendance_for_date(date_str)
189     elif choice == "4":
190         try:
191             sid_str = input("Enter student ID: ")
192             sid = int(sid_str)
193         except:
194             print("Invalid student ID!")
195             continue
196         print_summary_for_student(sid)
197     elif choice == "5":
198         print("Exiting attendance module...")
199         break
200     else:
201         print("Invalid choice. Try again.")
202
201
202
203
204 if __name__ == "__main__":
205     # Just directly call menu (also not modular)
206     main_menu()
207
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL AZURE
> TERMINAL
PS C:\Users\heman\OneDrive\Desktop\final exam ai> & C:/Python313/python.exe "c:/Users/heman/OneDrive/Desktop/final exam ai/attendance.py"

=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
3. View attendance for a date
4. View summary for a student
5. Exit
Enter your choice: 1
Student List
-----
ID: 1 | Name: Alice | Active
ID: 2 | Name: Bob | Active
ID: 3 | Name: Charlie | Inactive
ID: 4 | Name: Diana | Active

=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
3. View attendance for a date
4. View summary for a student
5. Exit
Enter your choice: 2
Enter date (YYYY-MM-DD): 2025-11-01
Enter student ID: 1
Enter status (P/A/L or Present/Absent/Late): P
Attendance marked for student: Alice

=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
3. View attendance for a date
4. View summary for a student
5. Exit
Enter your choice: 3
Enter date (YYYY-MM-DD): 2025-11-01
Attendance for 2025-11-01
-----
ID: 1 | Name: Alice | Status: P | Active
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL AZURE
> TERMINAL
PS C:\Users\heman\OneDrive\Desktop\final exam ai> & C:/Python313/python.exe "c:/Users/heman/OneDrive/Desktop/final exam ai/attendance.py"

Enter date (YYYY-MM-DD): 2025-11-01
Attendance for 2025-11-01
-----
ID: 1 | Name: Alice | Status: P | Active
ID: 2 | Name: Bob | Status: N/A | Active
ID: 3 | Name: Charlie | Status: N/A | Inactive
ID: 4 | Name: Diana | Status: N/A | Active

=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
Enter date (YYYY-MM-DD): 2025-11-01
Attendance for 2025-11-01
-----
ID: 1 | Name: Alice | Status: P | Active
ID: 2 | Name: Bob | Status: N/A | Active
ID: 3 | Name: Charlie | Status: N/A | Inactive
ID: 4 | Name: Diana | Status: N/A | Active

=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
=== ATTENDANCE MODULE (OLD) ===
1. Show all students
2. Mark attendance
3. View attendance for a date
4. View summary for a student
5. Exit
Enter your choice: 5
Exiting attendance module...
```

OBSERVATION:

The old attendance module contains many redundant conditions and repeated logic. Functions are poorly modular and handle multiple responsibilities.

Validation and boolean checks are unnecessarily complex.
The code needs simplification and refactoring for better readability and maintainability.

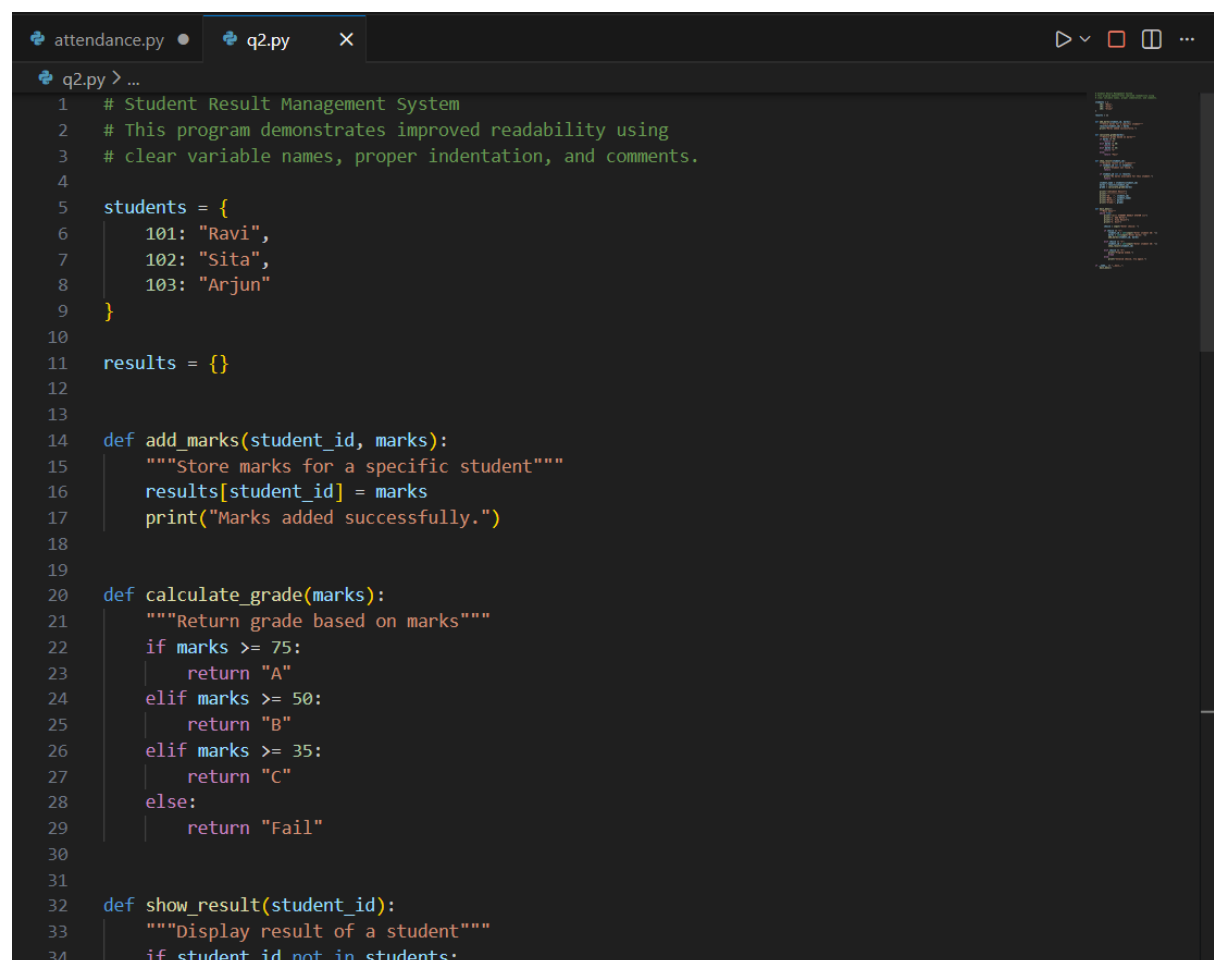
Q2: Improve readability

- Task 1: Ask AI to rewrite confusing variable names.
- Task 2: Improve indentation and comments

PROMPT:

Improve the readability of this code by renaming unclear variables, fixing indentation, and adding meaningful comments, while keeping the program's functionality the same.

CODE:



```
attendance.py • q2.py X
q2.py > ...
1 # Student Result Management System
2 # This program demonstrates improved readability using
3 # clear variable names, proper indentation, and comments.
4
5 students = {
6     101: "Ravi",
7     102: "Sita",
8     103: "Arjun"
9 }
10
11 results = {}
12
13
14 def add_marks(student_id, marks):
15     """Store marks for a specific student"""
16     results[student_id] = marks
17     print("Marks added successfully.")
18
19
20 def calculate_grade(marks):
21     """Return grade based on marks"""
22     if marks >= 75:
23         return "A"
24     elif marks >= 50:
25         return "B"
26     elif marks >= 35:
27         return "C"
28     else:
29         return "Fail"
30
31
32 def show_result(student_id):
33     """Display result of a student"""
34     if student_id not in students:
```

```
attendance.py • q2.py ×
q2.py > ...
32 def show_result(student_id):
33     print("Student not found.")
34     return
35
36     if student_id not in results:
37         print("No marks available for this student.")
38         return
39
40     student_name = students[student_id]
41     marks = results[student_id]
42     grade = calculate_grade(marks)
43
44     print("\nStudent Result")
45     print("-----")
46     print("ID   :", student_id)
47     print("Name :", student_name)
48     print("Marks:", marks)
49     print("Grade:", grade)
50
51
52
53
54 def main_menu():
55     """Main menu"""
56     while True:
57         print("\n=== STUDENT RESULT SYSTEM ===")
58         print("1. Add Marks")
59         print("2. View Result")
60         print("3. Exit")
61
62         choice = input("Enter choice: ")
63
64         if choice == "1":
65             student_id = int(input("Enter student ID: "))
66             marks = int(input("Enter marks: "))
67             add_marks(student_id, marks)
```

```
attendance.py • q2.py ×
q2.py > ...
54 def main_menu():
65     student_id = int(input("Enter student ID: "))
66     marks = int(input("Enter marks: "))
67     add_marks(student_id, marks)
68
69     elif choice == "2":
70         student_id = int(input("Enter student ID: "))
71         show_result(student_id)
72
73     elif choice == "3":
74         print("Program ended.")
75         break
76     else:
77         print("Invalid choice, try again.")
78
79
80 if __name__ == "__main__":
81     main_menu()
82
```

OUTPUT:

```
=== STUDENT RESULT SYSTEM ===
1. Add Marks
2. View Result
3. Exit
Enter choice: 1
Enter student ID: 101
Enter marks: 80
Marks added successfully.
```

```
=== STUDENT RESULT SYSTEM ===
1. Add Marks
2. View Result
3. Exit
Enter choice: 2
Enter student ID: 101
```

```
Student Result
-----
ID   : 101
Name : Ravi
Marks: 80
Grade: A
```

```
1. Add Marks
2. View Result
3. Exit
Enter choice: 2
Enter student ID: 101
```

```
Student Result
-----
ID   : 101
Name : Ravi
Marks: 80
Grade: A
```

```
Student Result
-----
ID   : 101
Name : Ravi
Marks: 80
Grade: A
```

Grade: A

```
=== STUDENT RESULT SYSTEM ===
1. Add Marks
2. View Result
3. Exit
Enter choice: 3
Program ended.
```

PS C:\Users\heman\OneDrive\Desktop\final exam ai> █

OBSERVATION:

The original code had unclear variable names and poor formatting.

Readability was improved by using meaningful names and proper indentation.

Helpful comments were added to explain the logic.

The program is now easier to understand while maintaining the same functionality